

Scheduling identical parallel machines with machine eligibility restrictions to minimize total weighted flowtime in automobile gear manufacturing

Ravindra Gokhale · M. Mathirajan

Received: 13 January 2011 / Accepted: 19 September 2011 / Published online: 6 October 2011
© Springer-Verlag London Limited 2011

Abstract In this paper, we address a scheduling problem for minimizing total weighted flowtime, observed in automobile gear manufacturing. Specifically, the bottleneck operation of the pre-heat treatment stage of gear manufacturing process has been dealt with in scheduling. Many real-life scenarios like unequal release times, sequence dependent setup times, and machine eligibility restrictions have been considered. A mathematical model taking into account dynamic starting conditions has been proposed. The problem is derived to be NP-hard. To approach the problem, a few heuristic algorithms have been proposed. Based on planned computational experiments, the performance of the proposed heuristic algorithms is evaluated: (a) in comparison with optimal solution for small-size problem instances and (b) in comparison with the estimated optimal solution for large-size problem instances. Extensive computational analyses reveal that the proposed heuristic algorithms are capable of consistently yielding near-statistically estimated optimal solutions in a reasonable computational time.

Keywords Scheduling · Parallel machines · Mathematical model · Heuristic algorithms

1 Introduction

This paper addresses a specific research problem of scheduling jobs on identical parallel machines to minimize the total weighted flowtime. The problem of scheduling of parallel machines has been addressed by many researchers in various forms. Having multiple resources with different capabilities in parallel is common in many industries like painting, plastics, semi conductor, chemical, and paper manufacturing [29]. The motivation for the paper comes from the preheat treatment stage of automobile gear manufacturing process.

The automobile gear manufacturing process is in general grouped into three distinct stages: preheat treatment (pre-HT), heat treatment (HT), and postheat treatment (post-HT). A contemporary practice observed among automobile manufacturing organizations is to outsource a part of gear manufacturing process to the subcontractors. Accordingly, the pre-HT and the HT stages of gear manufacturing are outsourced. For a particular type of gear, the same subcontractor performs these two stages. The post-HT stage, being the finishing stage and important from the quality aspect, is carried out by the automobile manufacturing organizations in-house. Thus, a major portion of gear manufacturing operations are carried out by subcontractors (known as *gear manufacturers*).

This study focuses on the scheduling issues related to these gear manufacturers in the pre-HT stage of gear manufacturing. The gear-shaving operation is the final operation of the pre-HT stage and is the bottleneck operation since it takes a relatively longer processing time

Electronic supplementary material The online version of this article (doi:10.1007/s00170-011-3653-3) contains supplementary material, which is available to authorized users.

R. Gokhale
Indian Institute of Management Indore,
Indore 453331, India
e-mail: ravindra@iimindr.ac.in

M. Mathirajan (✉)
Department of Management Studies, Indian Institute of Science,
Bangalore 560 012, India
e-mail: msdmathi@mgmt.iisc.ernet.in

compared to other operations of this stage. In this study, we focus only on the bottleneck operation, and hence all the remaining operations can be considered as a *black box*.

Generally, there is more than one machine available for processing the gear-shaving operation. The processing time for a job is independent of the machine used. This condition is known as having identical parallel machines, and is denoted by P . The complexity in scheduling this operation is due to jobs arriving at different points of time, that is, *unequal release times* (r_j), with different job weights (w_j)—representing priorities of jobs (*priorities* are generally decided based on the type of jobs, urgency of the job, etc.) and setup requirements. The amount of time required for setup of different jobs includes time for change of tools and tooling, and depends on both, the job that has been completed (that is predecessor), and the job that is to be processed (that is successor). This gives rise to the *sequence-dependent setup times* (s_{ij}). Additionally, due to large variety of automobile gears, some machines may not be capable of processing some type of jobs, thus imposing the *machine eligibility restrictions* (denoted by M_j).

The pre-HT stage, being the first stage of gear manufacturing process, is important to feed the subsequent downstream stages. That is, from the gear manufacturer's point of view, this being the first stage of gear manufacturing process, maximizing the throughput is important. In the presence of unequal release times and different job priorities, the objective of minimizing the total weighted flowtime has to be achieved, which can be considered as a suitable performance measure to achieve the purpose. The flowtime of a job is the total amount of time the job spends in the system. Thus if C_j denotes the completion time of a job and r_j denotes the release time of the job, then the flowtime of the job (denoted by F_j) can be given as: $F_j = C_j - r_j$. Consequently, the *weighted flowtime* of the job is given by $w_j F_j$. Thus, the problem considered here can be written in the standard three-field notation as of Graham et al. [12] as: $P/r_j, s_{ij}, M_j/\Sigma w_j F_j$.

The paper is organized as follows: In Section 2, we give a related review of the literature on the single-stage parallel machine problems. Following this, a formal statement of the problem and the underlying assumptions are stated in Section 3. A mathematical model is developed in the next section, that is, Section 4. Further, in Section 5, we propose some simple heuristic algorithms. Next, in Section 6, we discuss the computational experiments and analysis of results. Finally, in Section 7, we give our concluding remarks.

2 Related review

The problem of scheduling parallel machines has been studied widely in literature. It has been studied under the

concepts of either single-stage parallel machines or hybrid flow shops (flexible flowshops). Multistage flowshops with identical parallel machines at each stage are referred to as *hybrid flowshops* (example: [21]). The problem considered in this study focuses only on the bottleneck operation and hence has a single stage. Cheng and Sin [8] give a review of the single-stage parallel machine problem. They have classified the problems under three performance measures, namely completion time based, due date based, and flowtime based. Allahverdi et al. [1] provide a review specifically related to the setup considerations for various scenarios like single machine, parallel machines, flow shop, and job shop. Allahverdi et al. [2] again provide an updated review based on the same topic as above, for the papers published from the time since their last review till the date then. Sen et al. [32] give a state-of-the-art review on static scheduling of machines, where the objective function is due date based, precisely total tardiness and total weighted tardiness.

With respect to the area of the application dealt with in this study, there is a scant treatment to the scheduling aspect of a particular type of component-like gear. Most of the studies related to gear manufacturing (not necessarily automobile gears) deal with the technical aspect (example: [28] and [7]). Some studies based on the managerial aspects of gear manufacturing deal with a more general problems like dispatching policies (example: [15]), manufacturing systems (example: [26]) and so on. A study by Gokhale and Mathirajan [10] is the one so far that has specifically addressed a problem related to the scheduling of automobile gear manufacturing process. They have focused on scheduling of batch processors in the heat treatment stage of gear manufacturing.

Traditionally, parallel machine scheduling problems can be broadly classified based on machine characteristics (i.e., type of machines—identical, uniform, and unrelated), different job characteristics (release times—equal or unequal; setup considerations—sequence and/or machine dependent or independent; other constraints—precedence, breakdowns, and machine eligibilities) and finally scheduling objectives (completion time based or due date based).

There is a considerable amount of literature available on scheduling of parallel machines. Majority of the available literature is on scheduling of identical parallel machines. This is followed by the literature on unrelated parallel machines (example: [33]). However, most of the studies therein focus on equal release times. With regard to the setup time considerations, only few studies have considered sequence-dependent setup times. Within those, some studies have considered special cases like family setup times (example: [34]). Regarding the machine eligibility restrictions, there has been a scant treatment in the literature.

As far as the scheduling objectives are concerned, there is almost an equal amount of study on the traditional completion time based objectives (like make span, total completion time, total weighted completion time, etc.) and traditional due date based objectives (like tardiness, number of tardy jobs, earliness–lateness, etc.). For completion time-based objectives, a few studies have considered special job characteristics like time-dependent processing times which may be important in some industries like steel production and services like fire fighting or maintenance (example: [18]). Similarly for due date-based objectives, some studies consider common due window for a job, instead of the conventional due date (example: [14]).

For the problem considered in this study, we have simultaneously considered three jobs characteristics: *unequal release times*, *setup times*, and *machine eligibility* restrictions. From the literature review, it is observed that none of the study deals with all these three characteristics simultaneously. However, studies have considered two job characteristics simultaneously. Such studies can be regarded as closely related to the problem considered by

us. Table 1 provides a quick overview of the closely related studies.

From Table 1, it is observed that among the studies dealing with identical parallel machines, unequal release times, and setup considerations [19,24,25,27,31], only Kurz and Askin [19] and Nessah et al. [24] have considered completion time-based objective, but neither have considered machine eligibility restrictions in their study. Studies dealing with the most general case of unrelated parallel machines [16,22] both deal with due date-based objective. Similarly, the only study considering machine eligibility restrictions [13] has also considered a due date-based objective.

3 Problem statement and assumptions

The problem considered in this study can be formally stated as: scheduling identical parallel machines in presence of unequal release times, sequence-dependent setup times, and machine eligibility restrictions to minimize the total weighted flowtime.

Table 1 A quick review of the closely related studies

Researcher	Scheduling		
	Problem	Approach	Objective
Ovacik and Uzsoy [25]	Identical parallel machines, unequal release times, sequence-dependent setup times	A decomposition-based approach (<i>Rolling Horizon Procedure</i>)	3
Heady and Zhu [13]	Identical parallel machines, sequence-dependent setup times, machine eligibility restrictions	A two-stage heuristic procedure	4
Balakrishnan et al. [5]	Uniform parallel machines, unequal release times, sequence-dependent setup times	Mathematical formulation and solution, Suggest a solution approach based on Bender's decomposition for large-size problems	4
Sivrikaya-Şerifoglu and Ulusoy [31]	Identical parallel machines and uniform parallel machines, unequal release times, sequence-dependent setup times	Genetic algorithms	5
Kurz and Askin [19]	Identical parallel machines, unequal release times, sequence-dependent setup times	Simple heuristic algorithms and metaheuristics	1
Kim and Shin [16]	Unrelated parallel machines, unequal release times, sequence-dependent setup times	Restricted tabu search	3
Bilge et al. [6]	Uniform parallel machines, unequal release times, sequence-dependent setup times	Tabu search	6
Anghinolfi and Paolucci [3]	Uniform parallel machines, unequal release times, sequence-dependent setup times	Hybrid metaheuristic approach	6
Armentano and Filho [4]	Uniform parallel machines, unequal release times, sequence-dependent setup times	Greedy random search adaptive procedure (GRASP)	6
Kim et al. [17]	Uniform parallel machines, unequal release times, sequence-dependent setup times	Efficient methods for obtaining initial solution and metaheuristics	6
Logendran et al. [22]	Unrelated parallel machines, unequal release times, sequence-dependent setup times	Tabu search-based algorithms	7
Nessah et al. [24]	Identical parallel machines, unequal release times, sequence dependent setup times	Polynomial time lower bound procedure, heuristic algorithms, branch-and-bound procedure	2
Pfund et al. [27]	Identical parallel machines, unequal release times, sequence-dependent setup times	Extensions of the apparent tardiness cost with setups (ATCS) approach	7

Objectives: 1 = minimize makespan, 2 = minimize total weighted completion time, 3 = minimize maximum lateness, 4 = minimize earliness–tardiness costs, 5 = minimize total weighted earliness–tardiness penalty, 6 = minimize total tardiness, 7 = minimize total weighted tardiness

The various assumptions considered for the problem are as follows:

1. Jobs are released in the system at fixed and discrete time epochs.
2. Among the set of eligible machines, the processing time of a job is machine independent.
3. Setup time between a predecessor–successor pair is symmetric and machine independent. Symmetric means that the time required for changing the setup from job i to job j , is equal to the time required for changing the setup from job j to job i .
4. Multistep setup is not allowed. That is, if it is desired to change the setup from job i to job k , then it should be carried out directly without any intermediate setup corresponding to a job j .
5. All processing times and setup times are deterministic and known apriori.
6. Setup/processing of a job cannot be discontinued in between (non pre-emptive).
7. Setup of a job cannot be started before the job is released in the system.
8. Work-in-process inventory in front of the machines is unlimited.
9. Machine breakdowns are not considered.

4 A mathematical model

In this section, we develop a mathematical model for the problem. A nomenclature used in the process of developing the mathematical model is as follows:

Sets

- J Jobs (n jobs, with job numbers $1, 2, \dots, n$)
 M Machines (m machines, with machine numbers $1, 2, \dots, m$)

Parameters

- r_j Release time of a job j
 w_j Weight of a job j
 p_j Processing time of a job j
 s_{ij} Setup time from current setting of job i to subsequent setting of job j
 e_{jk} 1 if machine k is eligible to process job j ; 0 otherwise

Decision variables

- X_{ijk} 1 if job j is processed immediately after job i on machine k ; 0 otherwise

Dependent variables

- C_j Completion time of a job j

- P_{ijk} Processing time (including setup time) of job j when it is processed immediately after job i on machine k

In addition to the real-life parameters, decision variables, and dependent variables, the proposed mathematical model requires “dummy jobs” for proper implementation of the mathematical model. The logical reasons for introducing the dummy jobs are as follows:

- In addition to unequal release times, it is also expected to have dynamic starting conditions for the machines. This means that all the machines may not be available at the beginning of the scheduling window (as they may be busy processing the jobs from the previous window). Similar real-life conditions can be found in study by Logendran et al. [22].
- At the time the machine is first available for processing, it will have the setup of the job completed by it at that time. Thus before processing the first job of the current scheduling window, a sequence-dependent setup time is incurred depending on the predecessor–successor pair.

Accordingly, m (= number of machines) dummy jobs are introduced. Due to this, the set J of jobs, will now assume values $1, 2, \dots, m+n$. Out of these jobs, the jobs: $1, 2, \dots, m$ are the dummy jobs and the jobs: $m+1, m+2, \dots, m+n$ are the “real” jobs. The dummy jobs $1, 2, \dots, m$ are the first jobs on the respective m machines. In other words, for each of the m machine, the first job in the sequence is a dummy job. For the available n jobs, the sequence is to be determined and it will be such that the objective function value will be minimized. The proposed mathematical model is given as:

$$\text{Minimize } \sum_{j=m+1}^{m+n} w_j (C_j - r_j) \quad (1)$$

Subject to:

$$P_{ijk} = s_{ij} + p_j + (\text{Big}M) \times (1 - e_{ik}) + (\text{Big}M) \times (1 - e_{jk}) \\ \forall i \in J, \forall j \in J / \{1, 2, \dots, m\}, \forall k \in M \quad (2)$$

$$C_i + \sum_{k=1}^m (P_{ijk} X_{ijk}) + (\text{Big}M) \times \left(\sum_{k=1}^m X_{ijk} - 1 \right) \leq C_j \quad (3)$$

$$\forall i \in J; \forall j \in J / \{1, 2, \dots, m\}$$

$$r_j + \sum_{i=1}^{m+n} \sum_{k=1}^m (P_{ijk} X_{ijk}) \leq C_j \quad \forall j \in J / \{1, 2, \dots, m\} \quad (4)$$

$$\sum_{i=1}^{m+n} \sum_{k=1}^m X_{ijk} = 1 \quad \forall j \in J / \{1, 2, \dots, m\} \quad (5)$$

$$\sum_{j=m+1}^{m+n} \sum_{k=1}^m X_{ijk} \leq 1 \quad \forall i \in J \quad (6)$$

$$\sum_{a=1}^{m+n} X_{ajk} + \sum_{b=m+1}^{m+n} \sum_{h \in M/\{k\}} X_{jbh} \leq 1 \quad (7)$$

$$\forall j \in J/\{1, 2, \dots, m\}, \forall k \in M$$

$$X_{ijk} \in \{0, 1\} \quad \forall i \in J, \forall j \in J/\{1, 2, \dots, m\}, \forall k \in M \quad (8)$$

Objective (1) minimizes the total weighted flowtime over all the jobs excluding the first m dummy jobs. The weighted flowtime of job can be defined as the product of weight of the job and the difference between completion time and release time of the job. As mentioned before, the term *weight* in this study represents the job priority.

Constraint (2) is used to derive the value of the dependent variable which indicates processing time (including the setup time) of a job that immediately succeeds a particular job on a particular machine. This value is derived using processing time of the job under consideration, setup time between the pair of jobs, and eligibilities of the job under consideration and the immediate preceding job on a machine. The structure of the constraint is such that only when both the jobs in the pair are eligible for the machine, a realistic value is returned, else a $BigM$ value is returned ($BigM$ indicates a large positive integer). Constraint (3) is used to compute the completion time of a job and ensures that the jobs succeed each other on the machines. Constraint (4) ensures that completion time of a job on a machine is not less than the sum of release time of the job and the processing time (including the setup time) of the current job.

Constraint (5) is the assignment constraint and ensures that all available n jobs are processed on only one machine and are immediately preceded by only one job. Constraint (6) ensures that any job can have at most one successor. Constraint (7) takes care of the condition that if an available job succeeds some job on a particular machine, then it cannot immediately precede some other job on any of the remaining machines. Finally, constraint (8) assigns binary values to the decision variable.

Apart from the constraints (2) to (8) explained above, it is required to introduce a constraint indicating the completion time of the dummy jobs, for the implementation of the model:

$$C_j = p_j \quad \forall j \in J/\{m+1, m+2, \dots, m+n\} \quad (9)$$

Constraint (9) sets the completion time of the dummy jobs equal to the processing time of those jobs. This

processing time of the dummy jobs indicates the first time availability of the respective machines in the scheduling window.

A validation of the mathematical model is given in Gokhale and Mathirajan [9]. The model has been implemented using LINGO® and solved optimally for a numerical example with eight jobs and three machines.

The problem considered in this study considers the job characteristics like unequal release times, sequence-dependent setup times, and machine eligibility restrictions for identical parallel machines. Even the single-machine problem with unequal release times to minimize the total weighted completion time (usually referred to as $1/r_j/\Sigma C_j$) is known to be NP-hard in the strong sense [20]. This allows us to deduce that the problem $P/r_j, s_{ij}, M_j/\Sigma w_j F_j$ considered in this study is also NP-hard in the strong sense. Hence to address this problem, we focus on development of efficient heuristic algorithms, which is discussed in the next section.

5 Heuristic algorithms

In order to address the NP-hard problem, we now propose five heuristic algorithms. All these proposed heuristic algorithms incorporate *unforced idleness* in them. The concept of *unforced idleness* pertains to the situation in which a machine is kept idle even if there is a job waiting to be processed. This tactic is adopted in anticipation of some other jobs, such that the overall objective function will be minimized. The extent of implementation of the unforced idleness depends upon the amount of jobs pending to be processed among the available n jobs at a given time t . Thus, this refers to the state of the system at a given time instant.

Out of the five proposed heuristic algorithms, four heuristic algorithms are based on the combinations of the job characteristics: job weight, job processing time, job setup time, and machine eligibility. In addition, one heuristic algorithm based on a *look-ahead strategy* has been proposed. The development details of these five heuristic algorithms are discussed next. While development of the heuristic algorithms, a two-phase procedure is used: (a) preparatory steps—they include computations based on all the available n jobs. They are not dependent on the time and are carried out at the beginning of the schedule; and (b) iterative steps—they are carried out at each time instance t when a decision needs to be made.

5.1 Weight-processing time-heuristic algorithm

Preparatory steps: For each machine k

- (a) Identify the total number of jobs that are eligible for processing on a machine k . Denote it by count _{k} .

- (b) Compute the average job weight for the jobs that the machine k is capable of processing. Denote it by $average_weight_k$.

Iterative steps: Involves the following steps:

- Step 1 At the time t (starting from $t=0$), select a machine that is available for processing. If no machine is available, then set $t=t+1$ and repeat the step.
- Step 2 Identify the number of jobs that the selected machine k is eligible for processing among all the jobs that are still pending for processing (these jobs may not be released in the system up to time t). Denote it by $num_pending_k$.
- Step 3 Compute the ratio: $num_pending_k/count_k$ for the selected machine. The ratio will range from 0 to 1. Through a series of pilot experiments, different levels have been arrived at, within this range. The threshold values are finalized at 0.80 for level 1, 0.65 for level 2, 0.45 for level 3, 0.15 for level 4, and finally 0 for level 5.
- Step 4 Preparation of ordered job list.
- (a) From the available n jobs, identify the pending jobs that are released in the system up to time t and are eligible to be processed on the selected machine. Depending on the weight of a job, the value of $average_weight_k$ of the selected machine and the level of the ratio $num_pending_k/count_k$ for the selected machine at the time t ; further, identify only those jobs j whose w_j satisfy the condition as given in Table 2. Copy the identified jobs in a temporary job list.

The conditions given in Table 2 are arrived at through a series of pilot experiments. The logic here is that lower levels of the ratio pertain to early stages of scheduling and hence the scheduler can afford to impose stricter conditions and wait longer. Higher levels of ratio pertain to later stages of scheduling and the scheduler needs to relax the condition accordingly. Finally at the highest level, no condition is imposed, that is all jobs are considered further.

Table 2 Conditions for preparing temporary list of jobs by allowing unforced idleness for heuristic algorithms

Level of the ratio: $num_pending_k/count_k$ for the selected machine	Condition to be applied at the time t on a job j for including the job in the modified list of jobs
Level 1	$w_j \geq average_weight_k$
Level 2	$w_j \geq 0.65 \times average_weight_k$
Level 3	$w_j \geq 0.20 \times average_weight_k$
Level 4	$w_j \geq 0.10 \times average_weight_k$
Level 5	—

- (b) If the temporary list is empty, then discard this machine and go to step 1; else, prepare an ordered job list from the temporary list of jobs, based on nondecreasing p_j/w_j value (where, p_j is the processing time of the job).

Step 5 Job selection and information updating

- (a) Select the topmost job in the ordered job list and schedule it for the selected machine.
- (b) Update the information of the job data and the next availability of the machine. Discard the temporary list and the ordered job list.

Step 6 If all the available n jobs are scheduled, then calculate the total weighted flowtime of the schedule; otherwise, go to step 1.

It is to be noted that throughout this study, the different cutoffs for various levels (example: step 3 in the heuristic algorithm weight-processing time-heuristic algorithm (WPH) discussed above) are arrived at based on the pilot experiments. They may change based on the nature of inputs. The intention of the study is to provide solution methodologies that can be applied in general. Hence, when the nature of inputs changes, a new set of pilot experiments is required to decide upon the new cut-offs.

5.2 Weight-setup time-heuristic algorithm

This heuristic algorithm is similar to the WPH algorithm, except that in step 4(b), the ordered list of jobs is prepared based on nondecreasing s_{ij}/w_j value (where, s_{ij} is the setup time between the latest completed job i (predecessor) and the job j being considered (successor)).

5.3 Weight-processing time-setup time-heuristic algorithm

This heuristic algorithm is similar to the WPH algorithm, except that in step 4(b), the ordered list of jobs is prepared based on nondecreasing $(p_j + s_{ij})/w_j$ value.

5.4 Look-ahead heuristic algorithm

Preparatory step: Compute the average job weight considering all the available n jobs. Denote it by $average_weight$.

Iterative steps: Involves the following steps:

- Step 1 At the time t (starting from $t=0$), identify the number of jobs that are still pending for processing. (These jobs may not be released in the system up to time t). Denote it by $num_pending$.
- Step 2 Compute the ratio: $num_pending/n$. The ratio will range from 0 to 1. The five levels corresponding to this ratio are arrived at and finalized by a similar

procedure as explained in step 3 of the WPH algorithm.

Step 3 Preparation of the ordered job list:

This step is similar to the step 4 of the WPH algorithm except that eligibility of the jobs on any particular machine is not checked at this stage. Thus for this algorithm, $average_weight_k$ is replaced by $average_weight$ and the ratio “ $num_pending_k/count_k$ ” is replaced by “ $num_pending/n$ ”.

Step 4 Preparation of ordered machine list:

- (a) For the topmost job in the ordered job list, identify all those machines that are eligible for processing the job and which will be available for processing on or before time $(t + LAC)$, where LAC is the *look ahead criterion*. The identified machines are copied in a temporary list of machines. Based on a series of pilot experiments conducted, the LAC has been taken as 2 hours for our computational experiments.
- (b) If the temporary list of machines is empty, then discard the job, and repeat step 4(a) with the next job from the ordered job list prepared in step 3(b). If all the jobs in the ordered job list have been exhausted, with the temporary list of machines being empty then set $t=t + 1$, discard the ordered job list and go to step 1; else, prepare an ordered machine list from the temporary list of machines based on the time at which a machine will complete the job under consideration. This time at which a machine will complete the job under consideration is computed by taking the sum of the time at which the machine will be next available for processing, s_{ij} (i.e., the setup time between the latest completed job i —the predecessor and the job j being considered—the successor) and p_j (i.e., processing time of the job j being considered).

Step 5 Machine selection and information updating:

- (a) Select the machine which will complete the job earliest.
- (b) Update the information of the selected job’s data and the next availability of the selected machine. Discard the temporary lists and the ordered lists of jobs and machines.

Step 6 If all the available n jobs are scheduled, then calculate the total weighted flowtime of the schedule; otherwise go to step 1.

5.5 Weight-time-eligibility heuristic algorithm

Preparatory steps Involve the following steps:

Step 1 Using w_j and job processing time (p_j) of all the available n jobs, compute the average job weight

(μ_w), standard deviation of the job weight (σ_w), average job processing time (μ_p), and standard deviation of job processing time (σ_p).

Step 2 For each job j :

- (a) Compute the normalized weights and processing times. Denote it by NW_j and NP_j , respectively. In general, suppose A is a characteristic, for example job weight, and v is the value of that characteristic, then the value v of A is normalized to Nv by computing: $Nv = (v - \mu_A)/\sigma_A$, where μ_A and σ_A are the average and standard deviation respectively, of characteristic over a set of jobs.
- (b) Identify the number of machines on which the job is eligible to be processed. Denote this by $machine_count_j$.
- (c) Compute the normalized value of $machine_count_j$. The normalized range is between -1 to $+1$. If m is the total number of machines, then the normalized value of $machine_count$ of job j is computed as: $[m + 1 - 2 \times (machine - count_j)] / (m - 1)$. Denote it by $N machine_count_j$.
- (d) Compute the average sequence-dependent setup time and the standard deviation of the sequence dependent setup time using the s_{ij} values. Denote it by μ_{js} and σ_{js} , respectively.

Step 3 For each machine k , compute $count_k$ and $average_weight_k$ (as in the preparatory step of the heuristic algorithm WPH)

Step 4 Among the $count_k$ values of all the machines, determine the maximum value (denote it by max_count) and the minimum value (denote it by min_count)

Step 5 For each machine, compute the $adjusted_count_k$ as: $count_k - min_count + 1$

Step 6 Among the $adjusted_count_k$ values of all the machines determine the maximum value (denote it by $max_adjusted_count$) and the minimum value (denote it by $min_adjusted_count$)

Step 7 For each machine k , compute the N -job- $count_k$ as: If $max_count = min_count$, then the N -job- $count_k$ is zero; else it is, $[max_adjusted_count + 1 - (2 \times adjusted_count_k)] / [max_adjusted_count - 1]$

Iterative steps Involve the following steps:

Step 8 At the time t (starting from $t=0$), among all the machines that are available for processing; select the machine having the highest N -job- $count$ value. If no machine is available, then set $t=t+1$ and repeat the step.

Step 9 Determine the value of $num_pending_k$ for the selected machine.

- Step 10 Compute the ratio: $\text{num_pending}_k / \text{count}_k$ for the selected machine.
- Step 11 Calculation of weighted score of jobs.
- (a) This step is similar to the step 4a of the WPH algorithm.
 - (b) If the temporary list is empty, then discard this machine and go to step 8; else for each job j in the temporary list:
 1. Considering the current job setting i on the machine, compute the normalized setup time: $\text{NS}_{ij} = (s_{ij} - \mu_{js}) / \sigma_{js}$, based on the sequence-dependent setup time s_{ij} among the job i and job j .
 2. Compute the weighted score of the job as: $[0.3 \times \text{NW}_j] - [0.2 \times \text{NP}_j] + [0.2 \times N - \text{machine} - \text{count}_j] - [0.3 \times \text{NS}_{ij}]$ (These weights are based on a series of pilot experiments conducted)
- Step 12 Job selection and information updating.
- (a) Select a job with the highest weighted score.
 - (b) Update the information of the selected job's data and the next availability of the selected machine. Discard the temporary list.
- Step 13 If all the available n jobs are scheduled, then calculate the total weighted flowtime of the schedule; otherwise go to step 8.

All the five proposed heuristic algorithms were coded in MATLAB® 7.0 and run on a 2.40 GHz Pentium-4 processor with 1 GB RAM, and tested for implementation on numeric data. An illustration of workability of the proposed heuristic algorithms is given by Gokhale and Mathirajan [9]. In the next section, we discuss the computational experiments carried out for the evaluation of the proposed heuristic algorithms.

6 Computational experiments

In order to understand the performance of the proposed heuristic algorithms, well-designed computational experiments are needed. Accordingly, the evaluations of the proposed heuristic algorithms are carried out in two perspectives: one based on the comparison of the solution obtained from the heuristic algorithms with the *optimal solution*—for small-size problems, and the other one based on the comparison of the solution obtained from the heuristic algorithms with the *estimated optimal solution* (EOS)—for large-size problems (since optimal solution cannot be obtained for real-life large-size problems). The method of EOS presented in Rardin and Uzsoy [30] which is based on the Weibull distribution is used by us. Details of

this method are given in the [Appendix](#). For any instance, to arrive at an estimate of the optimal solution, we use the five solutions obtained from each of the proposed heuristic algorithm plus five more heuristic solutions. These additional five heuristic solutions are obtained by running each of the proposed heuristic algorithms without making any allowance for unforced idleness. We use the two standard performance measures: (a) average relative performance deviation (ARPD)—which gives an average case performance of the heuristic algorithms and (b) maximum relative performance deviation (MRPD)—which gives a worst case performance of the heuristic algorithms. Readers can refer Mathirajan and Sivakumar [23] for the computation details of these two performance measures.

For conducting the experiments, a proper experimental design needs to be planned. Based on the observations from the industry as well as literature available, we identified seven factors affecting the performance of the system. These seven factors are: number of jobs (n), number of machines (m), machine eligibility (e), job release time (r), job weight (w), job processing time (p), and job setup time (s). The details of the experimental design giving the levels of the different factors, number of configurations, and total number of problem instances for small-size problems as well as for large-size problems are given in Table 3.

Note from Table 3 that the levels for the factor *machine eligibility* are dependent on the corresponding level of the factor *number of machines*. The time units for *release time*, *processing time*, and *setup time* are taken to be hours. The scheduling window, i.e., range of release time of jobs is 1 day (0–23 h) for small-size problems and 1 week (0–167 h) for large-size problems. The range of processing time is between 16 and 40 h. Similarly, the range of setup time is between 2 and 8 h. Finally, the job weights range between 1 and 8.

As mentioned earlier, each of the proposed heuristic algorithm was implemented using MATLAB® (version 7.0) and run on a 2.40 GHz Pentium-4 processor with 1 GB RAM. For each of the 96 small-size problem instances, total weighted flowtime (TWFT) solutions were obtained for all the five proposed heuristic algorithms. Additionally, a LINGO® set code was developed to generate the proposed mathematical model for any given problem instance. For each of the 96 small-size problem instances, optimal solutions were obtained using the LINGO® set code. Table 4 gives the analysis of results in comparison to the optimal solution for 96 small-size problem instances based on the two performance measures: ARPD and MRPD.

The analysis of results in comparison to the optimal solution (Table 4) clearly indicates that except the heuristic algorithm weight-setup time-heuristic algorithm (WSH), all the remaining heuristic algorithms perform well. The

Table 3 Summary of the proposed experimental design

Factor	Small-size problem instances		Large-size problem instances	
	Number of levels	Values	Number of levels	Values
No. of jobs (n)	3	8; 10; 12	6	25; 50; 75; 100; 125; 150
No. of machines (m)	2	2; 3	3	2; 4; 6
Machine eligibility (e)	2	U[1,2]; U[1,m]	2	U[1,m/2]; U[1,m]
Job release time (r)	1	U[0,23]	1	U[0,167]
Job weight (w)	2	U[1,8]; U[1,2] or U[7,8] with probability 30% and U[3,6] with probability 70%	2	Same as small size problem instances
Job processing time (p)	2	U[16,40]; U[16,23] or U[33,40] with probability 40% and U[24,32] with probability 60%	2	Same as small size problem instances
Setup time between jobs (s)	2	U[2,8]; U[2,3] or U[7,8] with probability 70% and U[4,6] with probability 30%	2	Same as small size problem instances
Number of configurations	96 (i.e., $3 \times 2 \times 2 \times 1 \times 2 \times 2 \times 2 \times 1$)		288 (i.e., $6 \times 3 \times 2 \times 1 \times 2 \times 2 \times 2$)	
Instances per configuration	1		10	
Total number of instances	96 (i.e., 96×1)		2,880 (i.e., 288×10)	

average deviation is within 5% of the optimal solution for all the four heuristic algorithms: WPH, weight-processing time-setup time-heuristic algorithm (WPSH), look-ahead heuristic algorithm (LAH), and weight-time-eligibility heuristic algorithm (WTEH). However one should be aware that these results are based on only 96 problem instances and that too, small-size problem instances, and hence may not be generalized.

As mentioned earlier for evaluating the performance of the proposed heuristic algorithms for large-size problem instances, the key solution used in this study is the EOS. Ideally, to obtain an EOS, a large number of feasible

solutions are needed. Hence, we need to determine what feasible solutions can be used for each problem instance. For this purpose, a pilot study was conducted to compare solutions obtained using two types of solution sets to compute the EOS for a problem instance.

The first solution set for each problem instance consisted of the 10 solutions (out of which five solutions were obtained from the five proposed heuristic algorithms in this study plus five more solutions were obtained from variants of each of the five proposed heuristic algorithm—by not allowing unforced idleness). That is, the EOS was computed based on ten heuristic solutions. The second

Table 4 Performance of the proposed heuristic algorithms in comparison with optimal solution

Configuration	Number of instances	The ARPD score by proposed heuristic algorithms—indicates average performance					The MRPD score by proposed heuristic algorithms—indicates worst case performance				
		WPH	WSH	WPSH	LAH	WTEH	WPH	WSH	WPSH	LAH	WTEH
(n1,*,*,*,*)	32	3.32	7.47	3.20	3.30	3.98	10.37	51.62	8.95	10.37	10.00
(n2,*,*,*,*)	32	4.21	7.87	4.02	4.71	4.97	11.53	19.79	11.53	11.53	14.39
(n3,*,*,*,*)	32	4.38	8.36	4.35	4.77	5.22	11.48	26.82	11.68	15.06	23.46
(*,m1,*,*,*)	48	4.07	7.21	4.07	4.16	4.60	10.85	20.42	10.85	10.80	15.03
(*,m2,*,*,*)	48	3.87	8.59	3.64	4.35	4.85	11.53	51.62	11.68	15.06	23.46
(*,*,e1,*,*)	48	3.82	8.20	3.40	4.04	4.39	11.53	51.62	11.53	11.53	15.03
(*,*,e2,*,*)	48	4.12	7.60	4.31	4.48	5.06	11.48	26.82	11.68	15.06	23.46
(*,*,*,w1,*,*)	48	4.17	6.46	3.77	4.10	5.66	11.53	26.82	11.68	11.53	23.46
(*,*,*,w2,*,*)	48	3.76	9.34	3.95	4.42	3.79	10.69	51.62	11.14	15.06	12.74
(*,*,*,p1,*)	48	3.76	9.26	3.68	3.93	4.95	11.48	51.62	11.68	11.48	23.46
(*,*,*,p2,*)	48	4.18	6.54	4.03	4.59	4.49	11.53	19.79	11.53	15.06	14.39
(*,*,*,*,s1)	48	3.63	8.43	3.67	4.01	4.83	11.53	51.62	11.68	15.06	23.46
(*,*,*,*,s2)	48	4.31	7.37	4.04	4.51	4.61	10.69	19.99	11.14	10.69	10.98
(*,*,*,*,*)	96	3.97	7.90	3.86	4.26	4.72	11.53	51.62	11.68	15.06	23.46

solution set for each problem instance consisted of 100 feasible solutions (90 random solutions and the ten heuristic solutions as before).

The usable EOS was obtained for 85 out of the 96 small-size problem instances, using these two solution sets. It is to be noted that for the remaining 11 instances, we were not able to use the computed EOS value since the EOS value given by the equation was less than the optimal value. It was observed that on the whole, the EOS computed using the second solution set gave only marginally better results (that is, closer to the optimal solution), compared with the EOS computed using the first solution set. Hence, due to the ease of computation involved, it was decided to compute the EOS only based on the 10 heuristic solutions, for a given problem instance. The procedure and the summarized results of the relative percentage deviation scores of the two benchmark methods in comparison with the optimal solution is given in Gokhale and Mathirajan [9].

Comparison of the EOS results with the optimal solution (detailed results not presented here for the sake of brevity), indicated that for 72 out of 85 instances, that is, 85% of instances, the range of relative percentage deviation for the EOS from the optimal solution was [0, 5]. Barring just one instance, all the remaining 84 problem instances had the EOS within 10% relative percentage deviation from the optimal solution. Also, the average relative percentage

deviation over all the 85 instances was 2.62%. This shows the effectiveness of the EOS as a benchmark procedure.

Proceeding ahead, each of the 2,880 large-size problem instances generated as given in Table 3 were solved using each of the five proposed heuristic algorithms to get solutions for TWFT. In general, it was observed that the computational time required to obtain the heuristic solution for TWFT for each of the large-size problem instance considered in this study was meager.

For each of the 2,880 problem instances, using the 10 heuristic solutions for TWFT, the estimated optimal solution was computed and the comparison analysis was carried out. Table 5 gives the analysis of results in comparison to the estimated optimal solution for 2,875 large-size problem instances. Note that we were able to use the EOS comparison method only for 2,875 instances out of the total 2,880 problem instances. For the remaining five instances, the EOS could not be used due to one of the following reasons: (a) the EOS value given by the equation was negative for one instance, and (b) the EOS value given by the equation was higher than the solution given by at least one heuristic algorithm for four instances.

The analysis based on large-size problem instances (Table 5) reveals that among the four better heuristic algorithms obtained from previous analysis (that is, with respect to small size problem instances), the heuristic

Table 5 Performance of the proposed heuristic algorithms in comparison with the estimated optimal solution

Configuration	Number of instances	The ARPD score by proposed heuristic algorithms—indicates average performance					The MRPD score by proposed heuristic algorithms—indicates worst case performance				
		WPH	WSH	WPSH	LAH	WTEH	WPH	WSH	WPSH	LAH	WTEH
(n1,*,*,*,*)	476	9.52	12.76	9.35	7.87	8.88	78.82	73.11	78.82	56.35	62.08
(n2,*,*,*,*)	479	5.14	11.33	4.47	5.01	3.40	69.15	75.41	69.08	70.89	63.29
(n3,*,*,*,*)	480	4.39	10.98	3.45	4.50	1.10	27.61	36.65	26.35	28.47	23.36
(n4,*,*,*,*)	480	4.85	10.97	3.69	4.85	0.45	14.14	23.94	12.97	13.84	8.76
(n5,*,*,*,*)	480	5.52	11.07	4.01	5.50	0.30	10.70	20.93	9.88	10.64	3.06
(n6,*,*,*,*)	480	6.06	10.88	4.35	5.96	0.25	15.44	24.05	14.60	15.56	10.90
(*,m1,*,*,*)	960	5.13	8.03	2.81	5.21	1.32	32.75	46.86	34.09	36.84	36.02
(*,m2,*,*,*)	960	5.96	12.34	5.42	5.81	2.37	69.15	75.41	69.08	70.89	63.29
(*,m3,*,*,*)	955	6.64	13.62	6.43	5.82	3.48	78.82	73.11	78.82	55.73	62.08
(*,*,e1,*,*)	1440	5.88	11.37	4.83	5.72	2.27	78.82	73.11	78.82	55.73	62.08
(*,*,e2,*,*)	1435	5.94	11.28	4.93	5.50	2.51	69.15	75.41	69.08	70.89	63.29
(*,*,*,w1,*,*)	1437	5.47	10.75	4.49	5.10	2.64	78.82	75.41	78.82	70.89	63.29
(*,*,*,w2,*,*)	1438	6.34	11.91	5.27	6.13	2.13	56.74	66.85	56.74	56.35	53.02
(*,*,*,p1,*)	1436	6.20	12.25	5.22	5.77	2.56	78.82	73.11	78.82	56.35	62.08
(*,*,*,p2,*)	1439	5.61	10.41	4.54	5.46	2.21	69.15	75.41	69.08	70.89	63.29
(*,*,*,*,s1)	1438	5.70	11.35	4.77	5.42	2.38	78.82	73.11	78.82	56.35	62.08
(*,*,*,*,s2)	1437	6.11	11.31	4.99	5.81	2.40	69.15	75.41	69.08	70.89	63.29
(*,*,*,*,*)	2875	5.90	11.33	4.88	5.61	2.38	78.82	75.41	78.82	70.89	63.29

algorithm WTEH outperforms the other heuristic algorithms. Additionally, this heuristic algorithm improves to a great extent as the number of jobs increase. Since there is only one level for the factor of *release time*; a higher number of jobs will indicate a *congested system*. Thus we can infer that as the system becomes more congested, the heuristic algorithm WTEH performs the best among all the proposed heuristic algorithms. The fact that the performance of the heuristic algorithm WTEH goes on improving as the system becomes congested can be used to a great advantage by the decision makers.

Although it can be concluded that the WTEH heuristic algorithm outweighs all the other heuristic algorithms, the significance of relatively simpler heuristic algorithms like WPH, WSH, and WPSH also needs to be highlighted. It can be seen that when the system is not relatively congested, there is not much difference in the performance of different heuristic algorithms. In these cases, simpler heuristic algorithms will be easier to implement at a shop floor level from the foreman's or the supervisor's perspective. Also, these relatively simpler heuristic algorithms make use of lesser information as compared to the WTEH heuristic algorithm. Therefore in different scenarios where different amount and type of information is available, one can make use of the most suitable heuristic algorithm. If much of the information is available at the beginning of the scheduling horizon, which is highly possible in the current age where a good amount of information technology can be used, then obviously WTEH heuristic algorithm will give generally better results. It can still be noted that even WTEH heuristic algorithm does not require before hand, the exact release time of a job and thus will be very helpful in real-life scenario.

7 Conclusion

We have addressed a flowtime minimization scheduling problem encountered while processing the preheat treatment stage of automobile gear manufacturing process. Real-life characteristics job like unequal release times, sequence-dependent setup times, and machine eligibility restrictions have been considered. An integer linear programming model is developed which takes into account dynamic starting conditions for the machines. Efficient heuristic algorithms have been proposed for the problem under consideration. Although almost all the heuristic algorithms give good solutions; from the analysis of results from the computational experiments for real-life large-size problems, we can pinpoint the best performing heuristic algorithm, particularly under congested systems that are of greater concern. Yet, the variety of heuristic algorithms discussed in this paper will help decision making in

different scenarios where different amount and type of information is available. The proposed solution methodologies may not be kept restricted to the automobile gear manufacturing and can also be extended to similar other processes of different industries.

Acknowledgment The authors gratefully acknowledge the valuable comments and remarks of the anonymous referees that improved the quality of the paper.

Appendix

A procedure for EOS

The idea of statistical estimation techniques for optimal values based on Weibull distribution is explained here as per Rardin and Uzsoy [30].

Begin by letting $Z(1)$, $Z(2)$, . . . , $Z(N)$ be the n independent group minima, and sort them so that: $Z(1) \leq Z(2) \leq \dots \leq Z(n)$.

Then, the location parameter or optimal value a can be estimated as follows:

$$a = \left\{ Z(1) \times Z(n) - [Z(2)]^2 \right\} / \{ Z(1) + Z(n) - 2 \times Z(2) \}$$

This value of a is the EOS that is used for evaluation.

A reliable estimate of the true optimum would be valuable in itself, but Golden and Alt [11] introduce more certainty by computing a lower confidence limit Z_l which should be less than or equal to the optimal value Z^* with high probability. Specifically, they estimate Weibull scale parameter b from a as

$$b = Z[0.63 \times n + 1] - a$$

and then compute

$$Z_l = a - b$$

In theory, this interval should cover the optimal value Z^* with probability approximately $(1 - e^{-n})$.

References

1. Allahverdi A, Gupta JND, Aldowaisan T (1999) A review of scheduling research involving setup considerations. *Omega* 27 (2):219–239
2. Allahverdi A, Ng CT, Cheng TCE, Kovalyov MY (2008) A survey of scheduling problems with setup times or costs. *Eur J Oper Res* 187(3):985–1032
3. Anghinolfi D, Paolucci M (2007) Parallel machine total tardiness scheduling with a new hybrid metaheuristic approach. *Comput Oper Res* 34(11):3471–3490
4. Armentano VA, Filho MFF (2007) Minimizing total tardiness in parallel machine scheduling with setup times: an adaptive memory-based GRASP approach. *Eur J Oper Res* 183(1):100–114

5. Balakrishnan N, Kanet JJ, Sridhiran SV (1999) Early/tardy scheduling with sequence dependent setups on uniform parallel machines. *Comput Oper Res* 26(2):127–141
6. Bilge Ü, Kiraç F, Kurtulan M, Pekgün P (2004) A tabu search algorithm for parallel machine total tardiness problem. *Comput Oper Res* 31(3):397–414
7. Bouzakis KD, Lili E, Michailidis N, Friderikos O (2008) Manufacturing of cylindrical gears by generating cutting process: a critical synthesis of analysis methods. *CIRP Annals-Manuf Tech* 57(2):676–696
8. Cheng TCE, Sin CCE (1990) A state-of-the-art review of parallel-machine scheduling research. *Eur J Oper Res* 47(3):271–292
9. Gokhale R, Mathirajan M (2010) A supplement to scheduling identical parallel machines with machine eligibility restrictions to minimize total weighted flowtime in automobile gear manufacturing [online]. Department of Management Studies, Indian Institute of Science. Available from: <http://www.mgmt.iisc.ernet.in/~msdmathi>. Accessed 13 Jan 2011
10. Gokhale R, Mathirajan M (2011) Heuristic algorithms for scheduling of a batch processor in automobile gear manufacturing. *Int J Prod Res* 49(10):2705–2728
11. Golden BL, Alt FB (1979) Interval estimation of a global optimum for large combinatorial problems. *Nav Res Logist Q* 26(1):69–77
12. Graham RL, Lawler EL, Lenstra JK, Rinnooy Kan AHG (1979) Optimization and approximation in deterministic sequencing and scheduling: a survey. *Ann Discrete Math* 5(4):287–326
13. Heady RB, Zhu Z (1998) Minimizing the sum of job earliness and tardiness in a multimachine system. *Int J Prod Res* 36(6):1619–1632
14. Huang RH, Yang CL, Huang HT (2010) Parallel machine scheduling with common due windows. *J Oper Res Soc* 61(4):640–646
15. Hutchison J, Keong Leong G, Ward PT (1993) Improving delivery performance in gear manufacturing at Jeffrey Division of Dresser Industries. *Interfaces* 23(2):69–79
16. Kim CO, Shin HJ (2003) Scheduling jobs on parallel machines: a restricted tabu search approach. *Int J Adv Manuf Technol* 22(3–4):278–287
17. Kim S-I, Choi H-S, Lee D-H (2007) Scheduling algorithms for parallel machines with sequence-dependent set-up and distinct ready times: minimizing total tardiness. *Proceedings of the Institution of Mechanical Engineers, Part B. J Eng Manuf* 221(6):1087–1096
18. Kuo W-H, Hsu C-J, Yang D-L (2009) A note on unrelated parallel machine scheduling with time-dependent processing times. *J Oper Res Soc* 60(3):431–434
19. Kurz ME, Askin RG (2001) Heuristic scheduling of parallel machines with sequence-dependent set-up times. *Int J Prod Res* 39(16):3747–3769
20. Lawler EL, Lenstra JK, Rinnooy Kan AHG, Shmoys DB (1993) Sequencing and scheduling: algorithms and complexity. In: Graves SC, Rinnooy Kan AHG, Zipkin P (eds) *Handbooks in operations research and management science 4, logistics of production and inventory*. Elsevier, Amsterdam, pp 445–524
21. Lee G-C, Kim Y-D, Choi S-W (2004) Bottleneck-focused scheduling for a hybrid flowshop. *Int J Prod Res* 42(1):165–181
22. Logendran R, McDonell B, Smucker B (2007) Scheduling unrelated parallel machines with sequence-dependent setups. *Comput Oper Res* 34(11):3420–3438
23. Mathirajan M, Sivakumar AI (2006) Minimizing total weighted tardiness on heterogeneous batch processing machines with incompatible job families. *Int J Adv Manuf Technol* 28(9–10):1038–1047
24. Nessah R, Chu C, Yalaoui F (2007) An exact method for $Pm/sds, r_i/\sum_{i=1}^n C_i$ problem. *Comput Oper Res* 34(9):2840–2848
25. Ovacik IM, Uzsoy R (1995) Rolling horizon procedures for dynamic parallel machine scheduling with sequence-dependent setup times. *Int J Prod Res* 33(11):3173–3192
26. Pelagagge PM (1997) Advanced manufacturing system for automotive components production. *Ind Manag Data Syst* 97(8):327–334
27. Pfund M, Fowler JW, Gadkari A, Chen Y (2008) Scheduling jobs on parallel machines with setup times and ready times. *Comput Ind Eng* 54(4):764–782
28. Pohit G (2006) Application of virtual manufacturing in generation of gears. *Int J Adv Manuf Technol* 31(1–2):85–91
29. Rabadi G, Moraga RJ, Al-Salem A (2006) Heuristics for the unrelated parallel machine scheduling problem with setup times. *J Intell Manuf* 17(1):85–97
30. Rardin RL, Uzsoy R (2001) Experimental evaluation of heuristic optimization algorithms: a tutorial. *J Heuristics* 7(3):261–304
31. Sivrikaya-Şerifoglu F, Ulusoy G (1999) Parallel machine scheduling with earliness and tardiness penalties. *Comput Oper Res* 26(8):773–787
32. Sen T, Sulek JM, Dileepan P (2003) Static scheduling research to minimize weighted and unweighted tardiness: a state-of-the-art survey. *Int J Prod Econ* 83(1):1–12
33. Shim S-O, Kim Y-D (2007) Minimizing total tardiness in an unrelated parallel-machine scheduling problem. *J Oper Res Soc* 58(3):346–354
34. Shin HJ, Leon VJ (2004) Scheduling with product family set-up times: an application in TFT LCD manufacturing. *Int J Prod Res* 42(20):4235–4248